Catálogo de sentencias SQL con resultado

Cada ejemplo: qué se pide en lenguaje normal, la sentencia y lo que devuelve de verdad (las sentencias se ejecutaron al generar este PDF). SQLite, esquema del Taller mecánico.

Tabla	Columnas (PK subrayada en negrita, FK con →)
clientes	**id_cliente**, nombre, email (UNIQUE), telefono (admite NULL)
vehiculos	**matricula**, marca, modelo, anio, id_cliente → clientes
mecanicos	**id_mecanico**, nombre, especialidad, tarifa_hora (CHECK >= 0)
reparaciones	**id_reparacion**, matricula → vehiculos, id_mecanico → mecanicos, descripcion, fecha (texto AAAA-MM-DD), horas_trabajo, estado (PENDIENTE | EN_PROCESO | FINALIZADA)
piezas_reparacion	**id_pieza**, id_reparacion → reparaciones, nombre, cantidad (CHECK > 0), precio_unitario

Datos: la base de ejemplo (6 clientes, 8 vehículos, 4 mecánicos, 10 reparaciones, 12 piezas) más 3 filas añadidas para que se vean los casos «sin»: el cliente **Pablo Gil** (sin vehículos y con teléfono NULL), el vehículo **0001AAA** (sin reparaciones) y el mecánico **Raúl Vega** (sin reparaciones). Los ejemplos de la última sección (INSERT / UPDATE / DELETE) se ejecutan en orden y se acumulan.

A. SELECT básico: columnas, orden, límite

1. Todos los clientes ordenados alfabéticamente.

```
SELECT id_cliente, nombre, email
FROM clientes
ORDER BY nombre;
```

id_cliente	nombre	email
1	Ana Martín	ana.martin@example.com
2	Carlos Pérez	carlos.perez@example.com
4	David López	david.lopez@example.com
6	Elena Suárez	elena.suarez@example.com
3	Lucía García	lucia.garcia@example.com
5	María Rodríguez	maria.rodriguez@example.com
7	Pablo Gil	pablo.gil@example.com

2. Los 3 mecánicos mejor pagados.

```
SELECT nombre, especialidad, tarifa_hora
FROM mecanicos
ORDER BY tarifa_hora DESC
LIMIT 3;
```

nombre	especialidad	tarifa_hora
Nuria Díaz	Diagnóstico	50
Sofía Ramos	Electricidad	45
Luis Cabrera	Motor	40

3. Vehículos del más nuevo al más antiguo (si empatan, por marca).

```
SELECT matricula, marca, modelo, anio
FROM vehiculos
ORDER BY anio DESC, marca ASC;
```

matricula	marca	modelo	anio
8901HIJ	Peugeot	208	2023
7890GHI	Kia	Ceed	2022
4567DEF	Renault	Clio	2021
2345BCD	Toyota	Corolla	2020
5678EFG	Hyundai	i30	2019
1234ABC	Seat	Ibiza	2018
3456CDE	Volkswagen	Golf	2017
6789FGH	Ford	Focus	2016
0001AAA	Seat	Leon	2015

4. Los estados que existen en las reparaciones, sin repetir (DISTINCT).

```
SELECT DISTINCT estado
FROM reparaciones
ORDER BY estado;
```

estado
EN_PROCESO
FINALIZADA
PENDIENTE

5. Columna calculada con alias: subtotal de cada pieza (cantidad × precio), las 5 más caras.

```
SELECT nombre, cantidad, precio_unitario,
       cantidad * precio_unitario AS subtotal
FROM piezas_reparacion
ORDER BY subtotal DESC
LIMIT 5;
```

nombre	cantidad	precio_unitario	subtotal
Alternador	1	135	135
Batería	1	110	110
Pastillas freno	2	45	90
Aceite motor 1L	4	11	44
Aceite motor 1L	4	11	44

6. Paginación: la «página 2» de 3 clientes por página (LIMIT + OFFSET).

```
SELECT id_cliente, nombre
FROM clientes
ORDER BY id_cliente
LIMIT 3 OFFSET 3;
```

id_cliente	nombre
4	David López
5	María Rodríguez
6	Elena Suárez

B. Filtros con WHERE

7. Vehículos de Seat o Toyota (IN en vez de varios OR).

```
SELECT matricula, marca, modelo
FROM vehiculos
WHERE marca IN ('Seat', 'Toyota');
```

matricula	marca	modelo
1234ABC	Seat	Ibiza
2345BCD	Toyota	Corolla
0001AAA	Seat	Leon

8. Vehículos fabricados entre 2018 y 2021, ambos incluidos (BETWEEN).

```
SELECT matricula, marca, anio
FROM vehiculos
WHERE anio BETWEEN 2018 AND 2021
ORDER BY anio;
```

matricula	marca	anio
1234ABC	Seat	2018
5678EFG	Hyundai	2019
2345BCD	Toyota	2020
4567DEF	Renault	2021

9. Clientes cuyo nombre contiene «ía» (LIKE con % a ambos lados).

```
SELECT nombre
FROM clientes
WHERE nombre LIKE '%ía%';
```

nombre
Lucía García
María Rodríguez

10. Piezas cuyo nombre empieza por «Filtro» o contiene «aceite» (LIKE no distingue mayúsculas en ASCII).

```
SELECT id_pieza, nombre
FROM piezas_reparacion
WHERE nombre LIKE 'Filtro%' OR nombre LIKE '%aceite%';
```

id_pieza	nombre
1	Filtro aceite
2	Aceite motor 1L
8	Filtro aire
9	Aceite motor 1L

11. Reparaciones NO finalizadas.

```
SELECT id_reparacion, descripcion, estado
FROM reparaciones
WHERE estado <> 'FINALIZADA';
```

id_reparacion	descripcion	estado
4	Diagnóstico electrónico	EN_PROCESO
6	Reparación sistema eléctrico	PENDIENTE
7	Revisión de frenos	EN_PROCESO
8	Inspección de avería	PENDIENTE

12. AND + OR con paréntesis: pendientes o en proceso Y de más de 2 horas.

```
SELECT id_reparacion, descripcion, horas_trabajo, estado
FROM reparaciones
WHERE (estado = 'PENDIENTE' OR estado = 'EN_PROCESO')
  AND horas_trabajo > 2;
```

id_reparacion	descripcion	horas_trabajo	estado
6	Reparación sistema eléctrico	4	PENDIENTE

Sin los paréntesis, AND se evalúa antes que OR y el resultado cambia. Ante la duda, paréntesis.

13. Reparaciones de septiembre de 2026 (las fechas ISO en texto se comparan bien).

```
SELECT id_reparacion, fecha, descripcion
FROM reparaciones
WHERE fecha BETWEEN '2026-09-01' AND '2026-09-30'
ORDER BY fecha;
```

id_reparacion	fecha	descripcion
1	2026-09-10	Cambio de aceite y filtros
2	2026-09-11	Revisión alternador
3	2026-09-12	Sustitución pastillas de freno
4	2026-09-15	Diagnóstico electrónico
5	2026-09-18	Sustitución correa auxiliar

14. Clientes sin teléfono registrado (IS NULL; nunca «= NULL»).

```
SELECT id_cliente, nombre
FROM clientes
WHERE telefono IS NULL;
```

id_cliente	nombre
7	Pablo Gil

15. Mecánicos que cobran 40 o más y no son de Frenos.

```
SELECT nombre, especialidad, tarifa_hora
FROM mecanicos
WHERE tarifa_hora >= 40
  AND especialidad <> 'Frenos';
```

nombre	especialidad	tarifa_hora
Luis Cabrera	Motor	40
Sofía Ramos	Electricidad	45
Nuria Díaz	Diagnóstico	50

C. Agregados, GROUP BY y HAVING

16. Resumen de tarifas: cuántos mecánicos hay, mínima, máxima y media.

```
SELECT COUNT(*) AS mecanicos,
       MIN(tarifa_hora) AS minima,
       MAX(tarifa_hora) AS maxima,
       ROUND(AVG(tarifa_hora), 2) AS media
FROM mecanicos;
```

mecanicos	minima	maxima	media
5	36	50	41.9

17. Cuántas reparaciones hay en cada estado.

```
SELECT estado, COUNT(*) AS cantidad
FROM reparaciones
GROUP BY estado
ORDER BY cantidad DESC;
```

estado	cantidad
FINALIZADA	6
PENDIENTE	2
EN_PROCESO	2

18. Horas trabajadas y nº de reparaciones por mecánico.

```
SELECT id_mecanico,
       COUNT(*) AS reparaciones,
       SUM(horas_trabajo) AS horas
FROM reparaciones
GROUP BY id_mecanico
ORDER BY horas DESC;
```

id_mecanico	reparaciones	horas
2	3	8
1	3	7
3	2	4.5
4	2	2.5

19. HAVING: mecánicos con 3 o más reparaciones (HAVING filtra grupos; WHERE filtra filas).

```
SELECT id_mecanico, COUNT(*) AS reparaciones
FROM reparaciones
GROUP BY id_mecanico
HAVING COUNT(*) >= 3;
```

id_mecanico	reparaciones
1	3
2	3

20. Clientes que tienen más de un vehículo.

```
SELECT id_cliente, COUNT(*) AS vehiculos
FROM vehiculos
GROUP BY id_cliente
HAVING COUNT(*) > 1;
```

id_cliente	vehiculos
1	3
2	2

21. Dos columnas en el GROUP BY: reparaciones por mecánico y estado.

```
SELECT id_mecanico, estado, COUNT(*) AS n
FROM reparaciones
GROUP BY id_mecanico, estado
ORDER BY id_mecanico, estado;
```

id_mecanico	estado	n
1	FINALIZADA	3
2	FINALIZADA	2
2	PENDIENTE	1
3	EN_PROCESO	1
3	FINALIZADA	1
4	EN_PROCESO	1
4	PENDIENTE	1

22. Unidades y coste total de piezas de cada reparación.

```
SELECT id_reparacion,
       SUM(cantidad) AS unidades,
       SUM(cantidad * precio_unitario) AS coste
FROM piezas_reparacion
GROUP BY id_reparacion
ORDER BY coste DESC;
```

id_reparacion	unidades	coste
2	2	160
10	1	110
3	3	105
9	5	64
1	5	58.5
5	1	38
7	1	15
6	3	7.5

23. COUNT(DISTINCT): cuántos vehículos distintos han pasado por el taller.

```
SELECT COUNT(DISTINCT matricula) AS vehiculos_atendidos
FROM reparaciones;
```

vehiculos_atendidos
8

24. Media de horas por estado, redondeada.

```
SELECT estado, ROUND(AVG(horas_trabajo), 2) AS media_horas
FROM reparaciones
GROUP BY estado;
```

estado	media_horas
EN_PROCESO	1.75
FINALIZADA	2.25
PENDIENTE	2.5

25. Gasto total en piezas de todo el taller.

```
SELECT ROUND(SUM(cantidad * precio_unitario), 2) AS total_piezas
FROM piezas_reparacion;
```

total_piezas
558

D. JOIN: relacionar tablas

26. INNER JOIN: cada vehículo con el nombre de su propietario.

```
SELECT v.matricula, v.marca, c.nombre AS propietario
FROM vehiculos v
INNER JOIN clientes c ON c.id_cliente = v.id_cliente
ORDER BY c.nombre;
```

matricula	marca	propietario
1234ABC	Seat	Ana Martín
2345BCD	Toyota	Ana Martín
0001AAA	Seat	Ana Martín
3456CDE	Volkswagen	Carlos Pérez
8901HIJ	Peugeot	Carlos Pérez
5678EFG	Hyundai	David López
7890GHI	Kia	Elena Suárez
4567DEF	Renault	Lucía García
6789FGH	Ford	María Rodríguez

27. Cuatro tablas encadenadas: reparación, cliente, vehículo y mecánico.

```
SELECT r.id_reparacion, c.nombre AS cliente, v.matricula,
       m.nombre AS mecanico, r.estado
FROM reparaciones r
JOIN vehiculos v ON v.matricula   = r.matricula
JOIN clientes  c ON c.id_cliente  = v.id_cliente
JOIN mecanicos m ON m.id_mecanico = r.id_mecanico
ORDER BY r.id_reparacion;
```

id_reparacion	cliente	matricula	mecanico	estado
1	Ana Martín	1234ABC	Luis Cabrera	FINALIZADA
2	Ana Martín	2345BCD	Sofía Ramos	FINALIZADA
3	Carlos Pérez	3456CDE	Pedro Medina	FINALIZADA
4	Lucía García	4567DEF	Nuria Díaz	EN_PROCESO
5	Ana Martín	1234ABC	Luis Cabrera	FINALIZADA
6	David López	5678EFG	Sofía Ramos	PENDIENTE
7	María Rodríguez	6789FGH	Pedro Medina	EN_PROCESO
8	Elena Suárez	7890GHI	Nuria Díaz	PENDIENTE
9	Carlos Pérez	8901HIJ	Luis Cabrera	FINALIZADA
10	Carlos Pérez	3456CDE	Sofía Ramos	FINALIZADA

28. Contactar con los clientes que tienen reparaciones PENDIENTES (JOIN + filtro).

```
SELECT c.nombre, c.telefono, v.matricula, r.descripcion
FROM reparaciones r
JOIN vehiculos v ON v.matricula  = r.matricula
JOIN clientes  c ON c.id_cliente = v.id_cliente
WHERE r.estado = 'PENDIENTE';
```

nombre	telefono	matricula	descripcion
David López	600444555	5678EFG	Reparación sistema eléctrico
Elena Suárez	600666777	7890GHI	Inspección de avería

29. LEFT JOIN + COUNT: clientes con su nº de vehículos, incluidos los que tienen 0.

```
SELECT c.nombre, COUNT(v.matricula) AS vehiculos
FROM clientes c
LEFT JOIN vehiculos v ON v.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nombre
ORDER BY vehiculos, c.nombre;
```

nombre	vehiculos
Pablo Gil	0
David López	1
Elena Suárez	1
Lucía García	1
María Rodríguez	1
Carlos Pérez	2
Ana Martín	3

COUNT(v.matricula) ignora los NULL del LEFT JOIN y da 0. Con COUNT(*) Pablo Gil saldría con 1.

30. Anti-join: clientes SIN vehículos (LEFT JOIN + IS NULL en la tabla de la derecha).

```
SELECT c.id_cliente, c.nombre
FROM clientes c
LEFT JOIN vehiculos v ON v.id_cliente = c.id_cliente
WHERE v.matricula IS NULL;
```

id_cliente	nombre
7	Pablo Gil

31. Vehículos que nunca han sido reparados.

```
SELECT v.matricula, v.marca, v.modelo
FROM vehiculos v
LEFT JOIN reparaciones r ON r.matricula = v.matricula
WHERE r.id_reparacion IS NULL;
```

matricula	marca	modelo
0001AAA	Seat	Leon

32. Mecánicos con el nº de reparaciones y las horas totales (0 si no tienen).

```
SELECT m.nombre,
       COUNT(r.id_reparacion) AS reparaciones,
       COALESCE(SUM(r.horas_trabajo), 0) AS horas
FROM mecanicos m
LEFT JOIN reparaciones r ON r.id_mecanico = m.id_mecanico
GROUP BY m.id_mecanico, m.nombre
ORDER BY reparaciones DESC;
```

nombre	reparaciones	horas
Luis Cabrera	3	7
Sofía Ramos	3	8
Pedro Medina	2	4.5
Nuria Díaz	2	2.5
Raúl Vega	0	0

33. Coste de piezas de TODAS las reparaciones, con 0 para las que no llevan piezas.

```
SELECT r.id_reparacion, r.descripcion,
       COALESCE(SUM(p.cantidad * p.precio_unitario), 0) AS coste_piezas
FROM reparaciones r
LEFT JOIN piezas_reparacion p ON p.id_reparacion = r.id_reparacion
GROUP BY r.id_reparacion, r.descripcion
ORDER BY r.id_reparacion;
```

id_reparacion	descripcion	coste_piezas
1	Cambio de aceite y filtros	58.5
2	Revisión alternador	160
3	Sustitución pastillas de freno	105
4	Diagnóstico electrónico	0
5	Sustitución correa auxiliar	38
6	Reparación sistema eléctrico	7.5
7	Revisión de frenos	15
8	Inspección de avería	0
9	Mantenimiento preventivo	64
10	Cambio de batería	110

34. Gasto total en piezas por cliente (cadena de LEFT JOIN, sin perder clientes).

```
SELECT c.nombre,
       COALESCE(SUM(p.cantidad * p.precio_unitario), 0) AS gasto
FROM clientes c
LEFT JOIN vehiculos v          ON v.id_cliente    = c.id_cliente
LEFT JOIN reparaciones r       ON r.matricula     = v.matricula
LEFT JOIN piezas_reparacion p  ON p.id_reparacion = r.id_reparacion
GROUP BY c.id_cliente, c.nombre
ORDER BY gasto DESC;
```

nombre	gasto
Carlos Pérez	279
Ana Martín	256.5
María Rodríguez	15
David López	7.5
Lucía García	0
Elena Suárez	0
Pablo Gil	0

35. Auto-join: parejas de vehículos del mismo propietario (a.matricula < b.matricula evita repetir).

```
SELECT a.id_cliente, a.matricula AS vehiculo_1, b.matricula AS vehiculo_2
FROM vehiculos a
JOIN vehiculos b ON a.id_cliente = b.id_cliente
                AND a.matricula < b.matricula;
```

id_cliente	vehiculo_1	vehiculo_2
1	1234ABC	2345BCD
2	3456CDE	8901HIJ
1	0001AAA	1234ABC
1	0001AAA	2345BCD

E. Subconsultas

Una subconsulta es un SELECT dentro de otro. Va entre paréntesis y puede estar en el **WHERE** (con IN, EXISTS o comparación), en el **SELECT** (un valor por fila) o en el **FROM** (como si fuera una tabla).

36. IN: clientes que tienen al menos un vehículo.

```
SELECT nombre
FROM clientes
WHERE id_cliente IN (SELECT id_cliente FROM vehiculos);
```

nombre
Ana Martín
Carlos Pérez
Lucía García
David López
María Rodríguez
Elena Suárez

37. NOT IN: clientes sin vehículos.

```
SELECT nombre
FROM clientes
WHERE id_cliente NOT IN (SELECT id_cliente FROM vehiculos);
```

nombre
Pablo Gil

Si la subconsulta del NOT IN devuelve algún NULL, el resultado queda vacío. Con claves NOT NULL es seguro; si no, usa NOT EXISTS.

38. EXISTS: clientes con algún vehículo (la subconsulta se relaciona con la fila de fuera).

```
SELECT c.nombre
FROM clientes c
WHERE EXISTS (SELECT 1
              FROM vehiculos v
              WHERE v.id_cliente = c.id_cliente);
```

nombre
Ana Martín
Carlos Pérez
Lucía García
David López
María Rodríguez
Elena Suárez

39. NOT EXISTS: reparaciones sin ninguna pieza.

```
SELECT r.id_reparacion, r.descripcion
FROM reparaciones r
WHERE NOT EXISTS (SELECT 1
                  FROM piezas_reparacion p
                  WHERE p.id_reparacion = r.id_reparacion);
```

id_reparacion	descripcion
4	Diagnóstico electrónico
8	Inspección de avería

40. Subconsulta escalar: mecánicos que cobran más que la media.

```
SELECT nombre, tarifa_hora
FROM mecanicos
WHERE tarifa_hora > (SELECT AVG(tarifa_hora) FROM mecanicos);
```

nombre	tarifa_hora
Sofía Ramos	45
Nuria Díaz	50

41. El máximo con empates: mecánicos con la tarifa más alta.

```
SELECT nombre, tarifa_hora
FROM mecanicos
WHERE tarifa_hora = (SELECT MAX(tarifa_hora) FROM mecanicos);
```

nombre	tarifa_hora
Nuria Díaz	50

42. La pieza más cara (por precio unitario) de la tabla.

```
SELECT nombre, precio_unitario
FROM piezas_reparacion
WHERE precio_unitario = (SELECT MAX(precio_unitario) FROM piezas_reparacion);
```

nombre	precio_unitario
Alternador	135

43. Subconsulta en el SELECT (correlacionada): nº de reparaciones de cada vehículo.

```
SELECT v.matricula,
       (SELECT COUNT(*)
        FROM reparaciones r
        WHERE r.matricula = v.matricula) AS reparaciones
FROM vehiculos v
ORDER BY reparaciones DESC, v.matricula;
```

matricula	reparaciones
1234ABC	2
3456CDE	2
2345BCD	1
4567DEF	1
5678EFG	1
6789FGH	1
7890GHI	1
8901HIJ	1
0001AAA	0

44. Subconsulta en el FROM (tabla derivada): coste medio de una reparación con piezas.

```
SELECT ROUND(AVG(coste), 2) AS coste_medio
FROM (SELECT id_reparacion,
             SUM(cantidad * precio_unitario) AS coste
      FROM piezas_reparacion
      GROUP BY id_reparacion);
```

coste_medio
69.75

45. Subconsulta en HAVING: reparaciones con más unidades de piezas que la media.

```
SELECT id_reparacion, SUM(cantidad) AS unidades
FROM piezas_reparacion
GROUP BY id_reparacion
HAVING SUM(cantidad) > (SELECT AVG(u)
                        FROM (SELECT SUM(cantidad) AS u
                              FROM piezas_reparacion
                              GROUP BY id_reparacion));
```

id_reparacion	unidades
1	5
3	3
6	3
9	5

46. Clientes con el MAYOR número de vehículos (máximo calculado con subconsulta anidada).

```
SELECT id_cliente, COUNT(*) AS vehiculos
FROM vehiculos
GROUP BY id_cliente
HAVING COUNT(*) = (SELECT MAX(n)
                   FROM (SELECT COUNT(*) AS n
                         FROM vehiculos
                         GROUP BY id_cliente));
```

id_cliente	vehiculos
1	3

47. Dos niveles de IN: clientes que tienen alguna reparación pendiente.

```
SELECT nombre
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM vehiculos
    WHERE matricula IN (SELECT matricula
                        FROM reparaciones
                        WHERE estado = 'PENDIENTE'));
```

nombre
David López
Elena Suárez

48. Reparaciones hechas por mecánicos de la especialidad Motor o Frenos.

```
SELECT id_reparacion, descripcion, id_mecanico
FROM reparaciones
WHERE id_mecanico IN (SELECT id_mecanico
                      FROM mecanicos
                      WHERE especialidad IN ('Motor', 'Frenos'))
ORDER BY id_reparacion;
```

id_reparacion	descripcion	id_mecanico
1	Cambio de aceite y filtros	1
3	Sustitución pastillas de freno	3
5	Sustitución correa auxiliar	1
7	Revisión de frenos	3
9	Mantenimiento preventivo	1

49. La pieza más cara DE CADA reparación (correlacionada con MAX).

```
SELECT p.id_reparacion, p.nombre, p.precio_unitario
FROM piezas_reparacion p
WHERE p.precio_unitario = (SELECT MAX(p2.precio_unitario)
                           FROM piezas_reparacion p2
                           WHERE p2.id_reparacion = p.id_reparacion)
ORDER BY p.id_reparacion;
```

id_reparacion	nombre	precio_unitario
1	Filtro aceite	14.5
2	Alternador	135
3	Pastillas freno	45
5	Correa auxiliar	38
6	Fusible	2.5
7	Líquido frenos	15
9	Filtro aire	20
10	Batería	110

50. Coste total de cada reparación = mano de obra + piezas (subconsulta en el SELECT).

```
SELECT r.id_reparacion,
       r.horas_trabajo * m.tarifa_hora AS mano_obra,
       COALESCE((SELECT SUM(p.cantidad * p.precio_unitario)
                 FROM piezas_reparacion p
                 WHERE p.id_reparacion = r.id_reparacion), 0) AS piezas
FROM reparaciones r
JOIN mecanicos m ON m.id_mecanico = r.id_mecanico
ORDER BY r.id_reparacion;
```

id_reparacion	mano_obra	piezas
1	80	58.5
2	135	160
3	96.25	105
4	75	0
5	120	38
6	180	7.5
7	77	15
8	50	0
9	80	64
10	45	110

Cómo elegir: «que tienen» → IN / EXISTS · «que NO tienen» → NOT EXISTS o LEFT JOIN … IS NULL · «mayor que la media / igual al máximo» → subconsulta escalar · «por cada X, cuántos Y» → subconsulta en el SELECT o LEFT JOIN + GROUP BY.

F. CTE (WITH) y funciones ventana

51. CTE: reparaciones cuyo coste en piezas supera la media (la CTE evita repetir el cálculo).

```
WITH coste AS (
    SELECT id_reparacion, SUM(cantidad * precio_unitario) AS total
    FROM piezas_reparacion
    GROUP BY id_reparacion
)
SELECT r.id_reparacion, r.descripcion, c.total
FROM reparaciones r
JOIN coste c ON c.id_reparacion = r.id_reparacion
WHERE c.total > (SELECT AVG(total) FROM coste);
```

id_reparacion	descripcion	total
2	Revisión alternador	160
3	Sustitución pastillas de freno	105
10	Cambio de batería	110

52. Dos CTE encadenadas: coste total (mano de obra + piezas) y ranking de las 3 más caras.

```
WITH piezas AS (
    SELECT id_reparacion, SUM(cantidad * precio_unitario) AS total
    FROM piezas_reparacion
    GROUP BY id_reparacion
),
total AS (
    SELECT r.id_reparacion, r.descripcion,
           r.horas_trabajo * m.tarifa_hora + COALESCE(p.total, 0) AS coste
    FROM reparaciones r
    JOIN mecanicos m ON m.id_mecanico = r.id_mecanico
    LEFT JOIN piezas p ON p.id_reparacion = r.id_reparacion
)
SELECT * FROM total ORDER BY coste DESC LIMIT 3;
```

id_reparacion	descripcion	coste
2	Revisión alternador	295
3	Sustitución pastillas de freno	201.25
6	Reparación sistema eléctrico	187.5

53. ROW_NUMBER + PARTITION BY: la reparación más reciente de cada vehículo.

```
WITH t AS (
    SELECT matricula, id_reparacion, fecha,
           ROW_NUMBER() OVER (PARTITION BY matricula
                              ORDER BY fecha DESC, id_reparacion DESC) AS rn
    FROM reparaciones
)
SELECT matricula, id_reparacion, fecha
FROM t
WHERE rn = 1
ORDER BY matricula;
```

matricula	id_reparacion	fecha
1234ABC	5	2026-09-18
2345BCD	2	2026-09-11
3456CDE	10	2026-10-14
4567DEF	4	2026-09-15
5678EFG	6	2026-10-01
6789FGH	7	2026-10-05
7890GHI	8	2026-10-10
8901HIJ	9	2026-10-12

No se puede filtrar una ventana en el WHERE del mismo SELECT: se mete en una CTE (o subconsulta) y se filtra fuera.

54. RANK: ranking de mecánicos por tarifa (los empates comparten puesto).

```
SELECT nombre, tarifa_hora,
       RANK() OVER (ORDER BY tarifa_hora DESC) AS puesto
FROM mecanicos;
```

nombre	tarifa_hora	puesto
Nuria Díaz	50	1
Sofía Ramos	45	2
Luis Cabrera	40	3
Pedro Medina	38.5	4
Raúl Vega	36	5

55. Total acumulado de horas de trabajo por fecha (SUM como ventana).

```
SELECT id_reparacion, fecha, horas_trabajo,
       SUM(horas_trabajo) OVER (ORDER BY fecha, id_reparacion) AS acumulado
FROM reparaciones
ORDER BY fecha, id_reparacion;
```

id_reparacion	fecha	horas_trabajo	acumulado
1	2026-09-10	2	2
2	2026-09-11	3	5
3	2026-09-12	2.5	7.5
4	2026-09-15	1.5	9
5	2026-09-18	3	12
6	2026-10-01	4	16
7	2026-10-05	2	18
8	2026-10-10	1	19
9	2026-10-12	2	21
10	2026-10-14	1	22

56. Cada reparación comparada con la media de horas de SU mecánico (AVG … PARTITION BY).

```
SELECT id_reparacion, id_mecanico, horas_trabajo,
       ROUND(AVG(horas_trabajo) OVER (PARTITION BY id_mecanico), 2) AS media_mecanico
FROM reparaciones
ORDER BY id_mecanico, id_reparacion;
```

id_reparacion	id_mecanico	horas_trabajo	media_mecanico
1	1	2	2.33
5	1	3	2.33
9	1	2	2.33
2	2	3	2.67
6	2	4	2.67
10	2	1	2.67
3	3	2.5	2.25
7	3	2	2.25
4	4	1.5	1.25
8	4	1	1.25

Diferencia con GROUP BY: la ventana NO colapsa las filas; cada reparación sigue apareciendo y gana una columna extra.

G. Expresiones: CASE, COALESCE, texto, fechas, UNION

57. CASE: etiquetar la duración de cada reparación.

```
SELECT id_reparacion, horas_trabajo,
       CASE WHEN horas_trabajo >= 3 THEN 'larga'
            WHEN horas_trabajo >= 2 THEN 'media'
            ELSE 'corta'
       END AS duracion
FROM reparaciones
ORDER BY id_reparacion;
```

id_reparacion	horas_trabajo	duracion
1	2	media
2	3	larga
3	2.5	media
4	1.5	corta
5	3	larga
6	4	larga
7	2	media
8	1	corta
9	2	media
10	1	corta

58. CASE dentro de un agregado: contar de una vez las reparaciones por estado (una fila, tres columnas).

```
SELECT SUM(CASE WHEN estado = 'PENDIENTE'  THEN 1 ELSE 0 END) AS pendientes,
       SUM(CASE WHEN estado = 'EN_PROCESO' THEN 1 ELSE 0 END) AS en_proceso,
       SUM(CASE WHEN estado = 'FINALIZADA' THEN 1 ELSE 0 END) AS finalizadas
FROM reparaciones;
```

pendientes	en_proceso	finalizadas
2	2	6

59. COALESCE: mostrar «sin teléfono» cuando es NULL.

```
SELECT nombre, COALESCE(telefono, 'sin teléfono') AS telefono
FROM clientes
ORDER BY id_cliente;
```

nombre	telefono
Ana Martín	600111222
Carlos Pérez	600222333
Lucía García	600333444
David López	600444555
María Rodríguez	600555666
Elena Suárez	600666777
Pablo Gil	sin teléfono

60. Texto: concatenar con || y pasar a mayúsculas (UPPER), longitud (LENGTH).

```
SELECT UPPER(nombre) || ' <' || email || '>' AS contacto,
       LENGTH(nombre) AS letras
FROM clientes
WHERE id_cliente <= 3;
```

contacto	letras
ANA MARTíN <ana.martin@example.com>	10
CARLOS PéREZ <carlos.perez@example.com>	12
LUCíA GARCíA <lucia.garcia@example.com>	12

61. Fechas: reparaciones por mes con strftime.

```
SELECT strftime('%Y-%m', fecha) AS mes,
       COUNT(*) AS reparaciones
FROM reparaciones
GROUP BY mes
ORDER BY mes;
```

mes	reparaciones
2026-09	5
2026-10	5

62. Fechas: días que llevan esperando las pendientes (julianday resta fechas).

```
SELECT id_reparacion, fecha,
       CAST(julianday('2026-10-31') - julianday(fecha) AS INTEGER) AS dias_esperando
FROM reparaciones
WHERE estado = 'PENDIENTE';
```

id_reparacion	fecha	dias_esperando
6	2026-10-01	30
8	2026-10-10	21

63. GROUP_CONCAT: todas las matrículas de cada cliente en una sola celda.

```
SELECT c.nombre, GROUP_CONCAT(v.matricula, ', ') AS matriculas
FROM clientes c
JOIN vehiculos v ON v.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nombre
HAVING COUNT(*) > 1;
```

nombre	matriculas
Ana Martín	1234ABC, 2345BCD, 0001AAA
Carlos Pérez	3456CDE, 8901HIJ

64. UNION: una sola lista con clientes y mecánicos (mismas columnas; UNION quita repetidos, UNION ALL no).

```
SELECT nombre, 'cliente' AS tipo FROM clientes
UNION
SELECT nombre, 'mecánico' FROM mecanicos
ORDER BY nombre
LIMIT 6;
```

nombre	tipo
Ana Martín	cliente
Carlos Pérez	cliente
David López	cliente
Elena Suárez	cliente
Lucía García	cliente
Luis Cabrera	mecánico

65. «¿Existe?»: devuelve 1 o 0 (útil para un método booleano en Java).

```
SELECT EXISTS (SELECT 1
               FROM clientes
               WHERE email = 'ana.martin@example.com') AS existe;
```

existe
1

H. INSERT, UPDATE, DELETE (y los errores que verás como SQLException)

Estas sentencias **modifican** los datos (en una copia en memoria). En Java se ejecutan con `executeUpdate()`, que devuelve el nº de filas afectadas.

66. INSERT de una fila indicando las columnas (el id se autogenera).

```
INSERT INTO clientes (nombre, email, telefono)
VALUES ('Irene Soto', 'irene.soto@example.com', '600777888');
```

Filas afectadas: 1

Comprobamos el efecto:

```
SELECT id_cliente, nombre, telefono FROM clientes ORDER BY id_cliente DESC LIMIT 2;
```

id_cliente	nombre	telefono
8	Irene Soto	600777888
7	Pablo Gil	NULL

67. INSERT de varias filas de una vez.

```
INSERT INTO mecanicos (nombre, especialidad, tarifa_hora)
VALUES ('Marta Lozano', 'Chapa', 42),
       ('Iván Prieto',  'Pintura', 39.5);
```

Filas afectadas: 2

Comprobamos el efecto:

```
SELECT * FROM mecanicos ORDER BY id_mecanico DESC LIMIT 3;
```

id_mecanico	nombre	especialidad	tarifa_hora
7	Iván Prieto	Pintura	39.5
6	Marta Lozano	Chapa	42
5	Raúl Vega	Neumáticos	36

68. INSERT … SELECT: crear una reparación pendiente por cada vehículo sin reparaciones.

```
INSERT INTO reparaciones (matricula, id_mecanico, descripcion, fecha, horas_trabajo, estado)
SELECT v.matricula, 1, 'Revisión inicial', '2026-11-02', 1, 'PENDIENTE'
FROM vehiculos v
WHERE NOT EXISTS (SELECT 1 FROM reparaciones r WHERE r.matricula = v.matricula);
```

Filas afectadas: 1

Comprobamos el efecto:

```
SELECT id_reparacion, matricula, descripcion, estado FROM reparaciones ORDER BY id_reparacion DESC LIMIT 2;
```

id_reparacion	matricula	descripcion	estado
11	0001AAA	Revisión inicial	PENDIENTE
10	3456CDE	Cambio de batería	FINALIZADA

69. UPDATE con cálculo: subir un 10 % la tarifa de los mecánicos de Motor.

```
UPDATE mecanicos
SET tarifa_hora = ROUND(tarifa_hora * 1.10, 2)
WHERE especialidad = 'Motor';
```

Filas afectadas: 1

Comprobamos el efecto:

```
SELECT nombre, especialidad, tarifa_hora FROM mecanicos WHERE especialidad = 'Motor';
```

nombre	especialidad	tarifa_hora
Luis Cabrera	Motor	44

70. UPDATE con subconsulta: finalizar las reparaciones en proceso que ya tienen piezas.

```
UPDATE reparaciones
SET estado = 'FINALIZADA'
WHERE estado = 'EN_PROCESO'
  AND id_reparacion IN (SELECT id_reparacion FROM piezas_reparacion);
```

Filas afectadas: 1

Comprobamos el efecto:

```
SELECT id_reparacion, estado FROM reparaciones WHERE id_reparacion IN (4, 7);
```

id_reparacion	estado
4	EN_PROCESO
7	FINALIZADA

71. UPDATE con CASE: cambiar dos columnas según la fila.

```
UPDATE piezas_reparacion
SET precio_unitario = CASE WHEN precio_unitario >= 100 THEN precio_unitario * 0.9
                           ELSE precio_unitario END
WHERE id_pieza IN (3, 10, 12);
```

Filas afectadas: 3

Comprobamos el efecto:

```
SELECT id_pieza, nombre, precio_unitario FROM piezas_reparacion WHERE id_pieza IN (3, 10, 12);
```

id_pieza	nombre	precio_unitario
3	Alternador	121.5
10	Batería	99
12	Cableado	25

72. DELETE simple por clave.

```
DELETE FROM piezas_reparacion
WHERE id_pieza = 6;
```

Filas afectadas: 1

Comprobamos el efecto:

```
SELECT COUNT(*) AS piezas_restantes FROM piezas_reparacion;
```

piezas_restantes
11

73. DELETE con subconsulta: borrar los clientes que no tienen vehículos.

```
DELETE FROM clientes
WHERE NOT EXISTS (SELECT 1 FROM vehiculos v WHERE v.id_cliente = clientes.id_cliente);
```

Filas afectadas: 2

Comprobamos el efecto:

```
SELECT id_cliente, nombre FROM clientes ORDER BY id_cliente DESC LIMIT 3;
```

id_cliente	nombre
6	Elena Suárez
5	María Rodríguez
4	David López

74. ERROR de clave foránea: borrar un cliente que tiene vehículos (ON DELETE RESTRICT, con PRAGMA foreign_keys = ON).

```
DELETE FROM clientes
WHERE id_cliente = 1;
```

ERROR: FOREIGN KEY constraint failed

En Java llega como SQLException. Solución: borrar primero los hijos (piezas → reparaciones → vehículos) y al final el cliente.

75. ERROR de UNIQUE: insertar un email que ya existe.

```
INSERT INTO clientes (nombre, email)
VALUES ('Otra Ana', 'ana.martin@example.com');
```

ERROR: UNIQUE constraint failed: clientes.email

76. ERROR de CHECK: tarifa negativa.

```
INSERT INTO mecanicos (nombre, especialidad, tarifa_hora)
VALUES ('Error Test', 'Motor', -5);
```

ERROR: CHECK constraint failed: tarifa_hora >= 0

77. ERROR de NOT NULL: falta una columna obligatoria.

```
INSERT INTO clientes (nombre)
VALUES ('Sin Email');
```

ERROR: NOT NULL constraint failed: clientes.email

78. ERROR de FK al insertar: reparación para una matrícula que no existe.

```
INSERT INTO reparaciones (matricula, id_mecanico, descripcion, fecha, horas_trabajo, estado)
VALUES ('0000XXX', 1, 'Prueba', '2026-11-02', 1, 'PENDIENTE');
```

ERROR: FOREIGN KEY constraint failed